ARQUITECTURA

EN CAPAS, CLIENTE-SERVIDOR Y MONOLÍTICA

EN CAPAS

Patrón de diseño de software que permite organizalo por medio de capaz horizontales.



- Cada capa cumple una **unica responsabilidad**
- Una capa solo puede comunicarse con la **capa inferior inmediata**

Resuelve problemas, por ejemplo:

- Código espaguetti: No **mezcla** reglas del negocio y BBDD
- Facilita el **mantenimiento** para que no sea complejo realizar cambios
- Permite trabajo en equipo **sin pisarse el codigo**.

Para aplicaciones medianas o grandes con lógica de negocio compleja

Ventajas

- **Mantenibilidad**: Permite identificar la parte del código que hay que cambiar
- **Ecalabilidad**: Se puede modificar una capa sin cambiar todo lo demás
- **Trabajo grupal**: Varios desarrolladores tabajan simultáneamente sin pisarse el código

Desventajas

- En **proyectos pequeños**, muchas capas pueden generar **código innecesario**, aumentan la cantidad de llamadas entre componentes y **más esfuerzo**

CLIENTE-SERVIDOR

Modelo en que las tareas se dividen entre los clientes y se solicitan servicios por medio de interfaces.

Proceso:

- El cliente envía una **petición** mediante la red
- El servidor **procesa la solicitud** y realiza según la **lógica del negocio**
- Realiza las **consultas** necesarias a la BBDD.
- Envía una **respuesta** al cliente

Resuelve problemas, por ejemplo:

- **Inconsistencia de datos**: la gestión dispersa e ineficiente de recursos, concentrándose en una única fuente.
- Elimina la necesidad de los usuarios tengan **dispositivos con gran capacidad de almacenamiento**.

Permite la comunicación y el intercambio de información entre diferentes sistemas



Ventajas

- **Mantenibilidad**: Permite identificar la parte del código que hay que cambiar
- **Ecalabilidad**: Se puede modificar una capa sin cambiar todo lo demás
- **Trabajo grupal**: Varios desarrolladores tabajan simultáneamente sin pisarse el código

Desventajas

- En **proyectos pequeños**, muchas capas pueden generar **código innecesario**, aumentan la cantidad de llamadas entre componentes y **más esfuerzo**

MONOLÍTICA

Modelo en que los componentes principales de una aplicación están integrados.

Características:

- Todos los módulos son parte de la misma aplicación
- Las funciones comparten el mismo código base
- Cuando se actualiza la aplicación hay que desplegarla nuevamente